



Enhancing time series forecasting with Large Language Models (LLMs), particularly in predicting network outages and anomalies

By Daniel, Tony, Nicholas, Ethan



01

ABSTRACT

Abstract

Motivation

- Cybersecurity needs accurate anomaly detection.
- LSTM models lack context-awareness for evolving threats.

Problem

- LSTMs struggle to capture complex multivariate time-series patterns

Approach

- Used reprogrammed DistilGPT2 via TIME-LLM.
- Applied prompt-tuning and numeric patch tokenization.

Abstract

Dataset

- CICIDS2017 was too complex to implement with our model.
- Used a Python-generated synthetic dataset similar to that of the CICIDS dataset.

Results

- Compared to LSTM using MSE, MAE, F1-score.
- DistilGPT2 outperformed in generalization & context.

Future Takeaways

- LLMs are viable for time series forecasting.
- Future work: prompt tuning, token design, use of Mistral/Gemma.



02

INTRODUCTION

INTRODUCTION

Research Hypothesis: *Can fine-tuned LLMs outperform LSTMs in cybersecurity forecasting with contextual data and limited training examples?*

RESEARCH OBJECTIVE

Proving that reprogrammed Large Language Models (LLMs) can capture context than traditional LSTMs, enabling stronger forecasting of network threats with minimal training data in improving cybersecurity anomaly detection.

Problem Statement & Importance

Problem Statement

- Traditional LSTM models are effective in handling and identifying sequential patterns but struggles to understand contextual information which limits their detection effectiveness for any cybersecurity anomalies.

Why This Research Matters

- Cybersecurity threats are becoming more advanced, so detection models must adapt to new and unknown attacks.
- Only rely on sequential data is no longer sufficient
- This research explores how LLMs can use context to improve forecasting and threat detection.

Challenges

Lack of Contextual Awareness

- LSTMs perform well with sequences but fail to capture broader contextual cues critical for cybersecurity threats.

Poor Adaptability to New Threats

- Struggle to detect novel or unseen attack patterns due to limited flexibility.

Difficulty with Complex DataSets

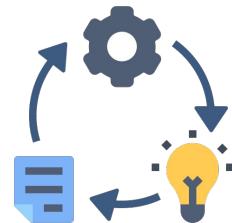
- Large and imbalanced datasets (e.g., CICIDS2017) reduce model accuracy and robustness.

Limited Generalization

- Require significant retraining to adapt to different datasets or environments.



Solution & Contributions



Solution

- Developed a forecasting approach using reprogrammed LLMs (e.g., DistilGPT2) with prompt engineering and tokenization.
- Converted numeric patches to dense 'pseudo-tokens' via linear projection.
- Built on the TIME-LLM framework to enable contextual reasoning in anomaly detection.
- Used a Python-generated synthetic dataset to overcome challenges with high-dimensional CICIDS2017 data.

Contributions

- Comparative Study: Evaluated performance between LSTM and LLM models using MSE, MAE, F1-score, Precision, and Recall.
- Improved Generalization: Demonstrated that LLMs better handle unseen attack patterns and adapt to complex data.
- Practical Framework: Validated the feasibility of using LLMs for time series forecasting in cybersecurity.
- Future Pathways: Proposed enhancements including prompt optimization and testing advanced LLMs (e.g., Mistral, Gemma).



03

RELATED WORKS

Related Works

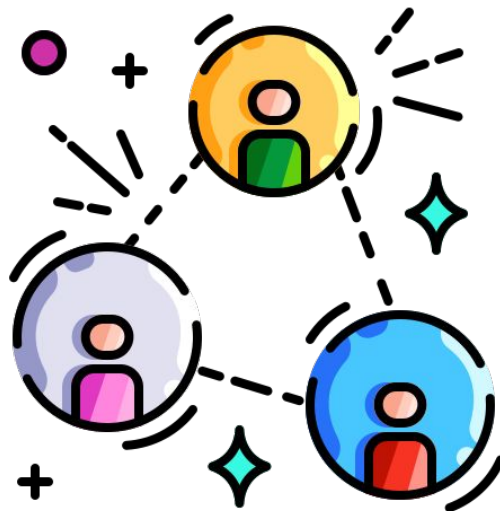
Traditional models

ARIMA:

- Struggles with multivariate and non-linear data
- Perfect for simple and univariate time series
- Uses lagged values for prediction.
- Not ideal for long-term patterns.

LSTMs

- Better than traditional RNN
- Weak with large, multivariate dataset
- Limited scalability for complex data



Related Works

Large Language Models

LLaMA 2 7b and LLaMA 2 13b

- Outperformed by Mistral in math and MMLU
- Not optimised for time-series data but adaptable
- Strong at reasoning and language tasks.

Gemma 7B and Mistral 7B

- Outperforms LLaMA-2 in mathematical reasoning
- Outperforms LLaMA-2 in mathematical problem solving

Benchmark	metric	LLaMA-2		Mistral	Gemma	
		7B	13B	7B	2B	7B
MMLU	5-shot, top-1	45.3	54.8	62.5	42.3	64.3
HellaSwag	0-shot	77.2	80.7	81.0	71.4	81.2
PIQA	0-shot	78.8	80.5	82.2	77.3	81.2
SIQA	0-shot	48.3	50.3	47.0*	49.7	51.8
Boolq	0-shot	77.4	81.7	83.2*	69.4	83.2
Winogrande	partial scoring	69.2	72.8	74.2	65.4	72.3
CQA	7-shot	57.8	67.3	66.3*	65.3	71.3
OBQA		58.6	57.0	52.2	47.8	52.8
ARC-e		75.2	77.3	80.5	73.2	81.5
ARC-c		45.9	49.4	54.9	42.1	53.2
TriviaQA	5-shot	72.1	79.6	62.5	53.2	63.4
NQ	5-shot	25.7	31.2	23.2	12.5	23.0
HumanEval	pass@1	12.8	18.3	26.2	22.0	32.3
MBPP [†]	3-shot	20.8	30.6	40.2*	29.2	44.4
GSM8K	maj@1	14.6	28.7	35.4*	17.7	46.4
MATH	4-shot	2.5	3.9	12.7	11.8	24.3
AGIEval		29.3	39.1	41.2*	24.2	41.7
BBH		32.6	39.4	56.1*	35.2	55.1
Average		47.0	52.2	54.0	44.9	56.4

Figure 1. Benchmark comparison of LLaMA-2 Mistral and Gemma (Patel 2024)

Related Works

Emerging Trend

- LLMs like *DistilGPT2* are being reprogrammed for time series using prompt tuning (e.g., TIME-LLM).
- They adapt to new tasks without full retraining.

Challenges

- Tokenizing numerical data.
- Generalising across tasks.
- High compute requirements.

Future Directions

- Test stronger models like *Mistral 7B* or *Gemma* for better performance.
- Compare model training epochs to measure efficiency.

Research Gap

- Few-shot LLM forecasting for cybersecurity is still underexplored.



04

PROBLEM FORMULATION

Problem Statement & Motivation

Research gaps

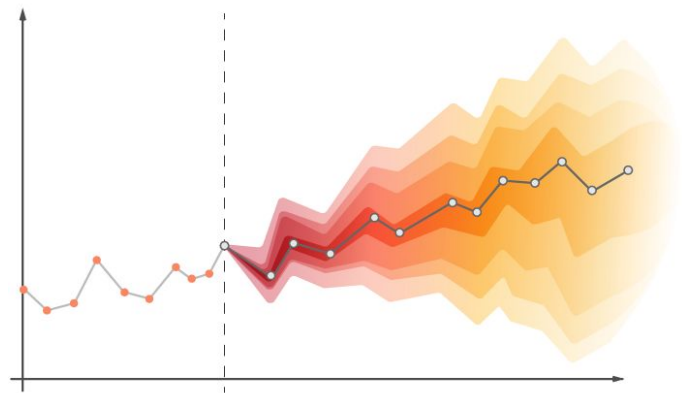
- Traditional LSTM lacks on contextual understanding
- LSTM cannot integrate real time contextual data

Solution

- Reprogramming LLMs for time series forecasting to bypass these ongoing limitations via the Time-LLM technique, numeric patch projection and prompt-tuning.

Motivation

- Cybersecurity time series forecasting prevents expensive network outages/ anomalies
- Re-programmed LLMs provide a dynamic, context-aware framework that transfers across domains beyond cybersecurity.



Problem Constraints

Incompatibility issues

- LLMs not built for time-series, so we re-programmed DistilGPT-2 with numeric patch projection + prompt-tuning instead of raw text tokens.

Hardware Bottlenecks

- Slow training on basic GPUs leads to limited accuracy/epochs.

Interpretability & generalisation

- Outputs must be clear to detect new attacks and requires good prompts design/model fine tuning.

Mitigation strategies

- Use smaller dataset for faster training

Tokenization

Converting sequential data into token-like dense vectors compatible with GPT-2

Scope & Limitations

Scope

- Focus mainly on cybersecurity anomaly detection
- Using a pre-trained DistilGPT-2 model re-programmed for time-series (no task-specific pre-training required)
- Framework is scalable with better computational resources

Limitations

- Requiring clear prompt for minimising misinterpretations
- Network datasets often requires longer training times.
- Preprocessing balances dataset size versus key features for accuracy.

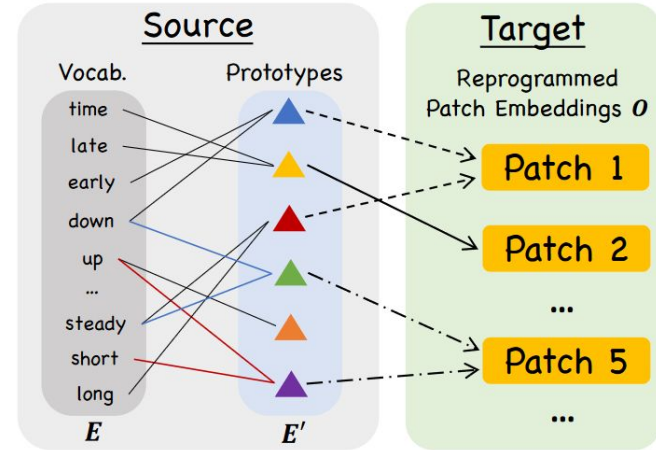


05

PROPOSED SOLUTIONS

High Level Idea

- Solution aims to leverage the contextual understanding properties of Large Language Models (LLMs) for time-series forecasting, specifically in cybersecurity
- Although LLMs are pre-trained to process natural language, they can also handle structured numeric inputs once those inputs are converted to token-like embeddings.
- Idea is to transform numerical time-series cybersecurity data into compact embedding “tokens” that encode local patterns in each patch.
- The LLM can process this quantitative data to predict future values, without altering its underlying architecture

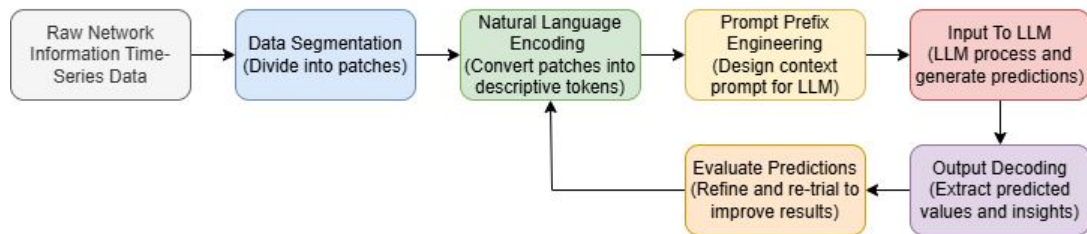


Transforming the input patch into a language task. Image by M. Jin, S. Wang, L. Ma, Z. Chu, J. Zhang, X. Shi, P. Chen, Y. Liang, Y. Li, S. Pan, Q. Wen from [Time-LLM: Time Series Forecasting by Reprogramming Large Language Models](#)

Methodology

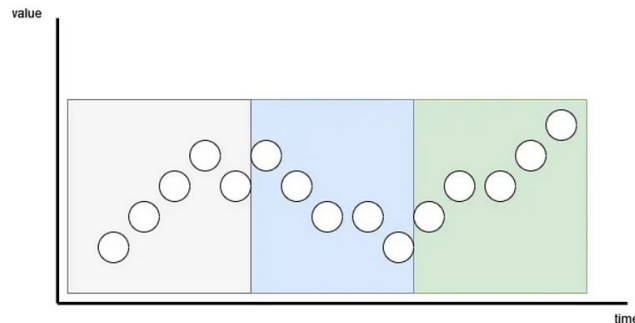
1. **Data Segmentation** and **Natural Language Encoding**
2. **Prompt Prefix Engineering** and **Model Input**
3. **Output Decoding** and **Evaluation**

Cybersecurity Time-Series Forecasting by Reprogramming LLMs



Data Segmentation and Numeric Token Projection

- Continuous time series data is segmented into “patches”
- Each patch is linearly projected into a dense embedding vector (“numeric token”) that captures its multivariate values and local trend.
- This learned projection lets the frozen GPT-2 backbone consume time-series patches as if they were text tokens, without altering its transformer layers (Jin et al., 2024).
- E.g. a sharp decline maps to a unique 768-dim vector that the model learns to associate with “downward movement.”



Visualizing patching. Here, we have a sequence of 15 timesteps, with a patch length of 5 and a stride of 5

Input : Time Series $X = [x_1, x_2, \dots, x_n]$
Output: Sequence of numeric tokens $E = [e_1, e_2, \dots]$

1. Split X into patches $p_1, p_2, \dots$ of length k
2. For each patch p_i :
 - a. Flatten / normalise values
 - b. Project to embedding $e_i = W \cdot p_i + b$
3. Return embedding sequence E

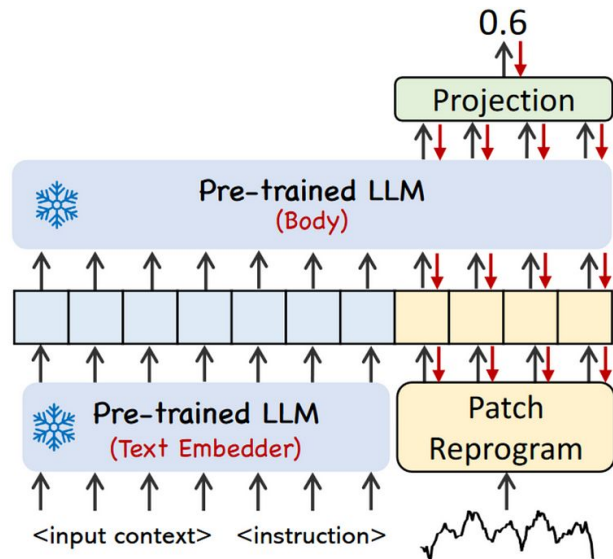
Prompt Prefix Engineering and Model Input

- A intently engineered prompt prefix is prepended to the sequence of language tokens
- The prompt aims to provide contextual information to the LLM about its role and the nature of the data
- Prompt engineering applies researched prompt patterns to customize and enhance LLM capabilities (White et al., 2023)

Prompt Segment	Prompt Prefix used in Solution
Name and Classification	Time-series Forecasting in Cybersecurity for anomaly detection and outage prevention. Provide an output of future predicted values in natural language tokens and numerical data
The Intent and Context	Cybersecurity is an ever-evolving problem and context is important to keep up with advancing attacks. Each data point consists of if an attack was present and 7 data features ... Below is information about the input time series:
Structure and Key Ideas	Predict the next <H> steps given the previous <T> steps with the patches of network data attached ...

Output Decoding and Evaluation

- The LLM outputs predicted future values in natural language tokens which can be decoded
- The prediction after decoding is compared to the actual observed values in the training set and evaluated based on Mean Absolute Error (MAE), Mean Squared Error (MSE), accuracy, precision and recall
- This evaluation will guide alterations that can be made to the prompt or encoding strategy and improve these metrics



The prompt prefix and reprogrammed patches are sent to the LLM. The final step is a linear projection to get the final predictions. Image by M. Jin, S. Wang, L. Ma, Z. Chu, J. Zhang, X. Shi, P. Chen, Y. Liang, Y. Li, S. Pan, Q. Wen from Time-LLM: Time Series Forecasting by Reprogramming Large Language Models

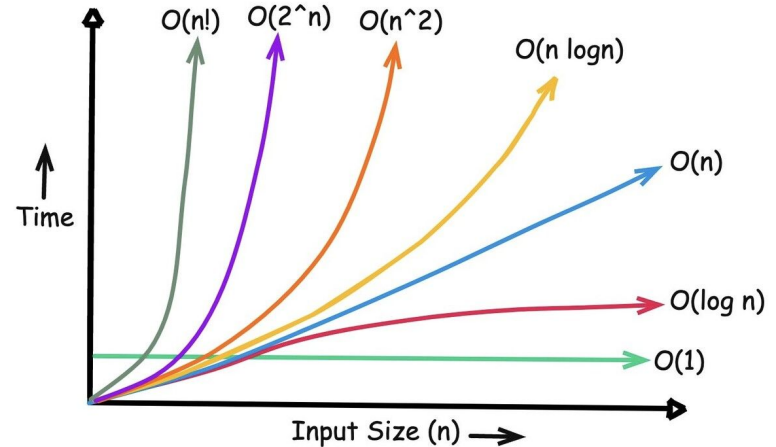
Complexity Analysis

- An LLM approach to time-series forecasting introduces *high computational complexity* and a *hardware bottleneck* if this was to be applied at a large scale due to exponential complexity
- **Data Segmentation & Output Decoding:** Assuming N points of data and patch size P the encoding process is relative to the number of patches

Complexity: $O(\frac{N}{P})$.

- **LLM Prediction Complexity:** Transformer based LLMs have time complexity of $O(P^2)$ where P is the prompt length in this case is the prefix and sequence of language tokens

Complexity: $O((P + \frac{N}{P})^2)$.





06

EXPERIMENTS

Experimental Setup

- **Dataset:** Synthetic network-traffic stream based upon real life datasets such as the cicits dataset. Contains 15,000 samples. Consists of 3 numeric features (bandwidth, latency, packet-loss) and 4 seasonal encodings. Train/Validation/Test split 70 / 10 / 20 %.
- **Software environments and Hardware:** Google Colab, NVIDIA Tesla T4 GPU.
- **Model:** Time-LLM based on DistilGPT2 running on 30 epochs. A 2-layered LSTM running on 10 epochs.



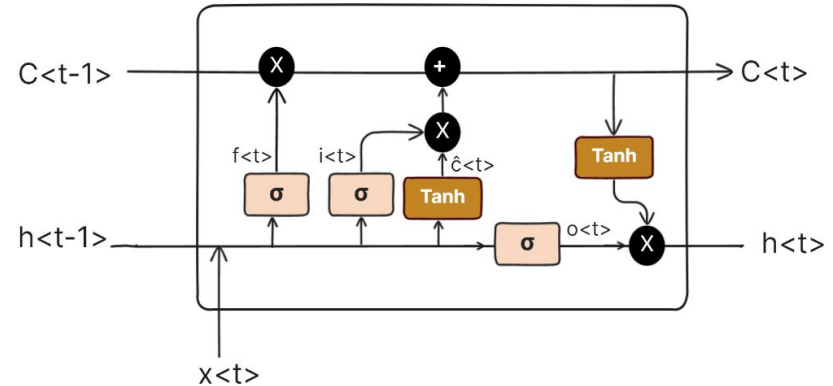
Metrics

Metric	Description
Precision	Measure of how many predicted positives are actually true positives
Recall	Measure of how many actual positives are correctly identified
F1 Score	Combines precision and recall into a single metric; especially valuable when classes are imbalanced
Mean Squared Error (MSE)	The average of the squared errors; makes larger errors more prominent
Mean Absolute Error (MAE)	Measures the average magnitude of errors between predicted and actual values

Baselines

- The baseline for our project was the 2-layered LSTM
- We chose the LSTM as the baseline due to its popularity for use in network-traffic & KPI forecasting.
- The LSTM was trained over 10 epochs and is made of 2 stacked layers.

LSTM Architecture



Results & Analysis

Time-LLM – MSE: 47.3013, MAE: 4.5619

LSTM – MSE: 54.0995, MAE: 4.8661

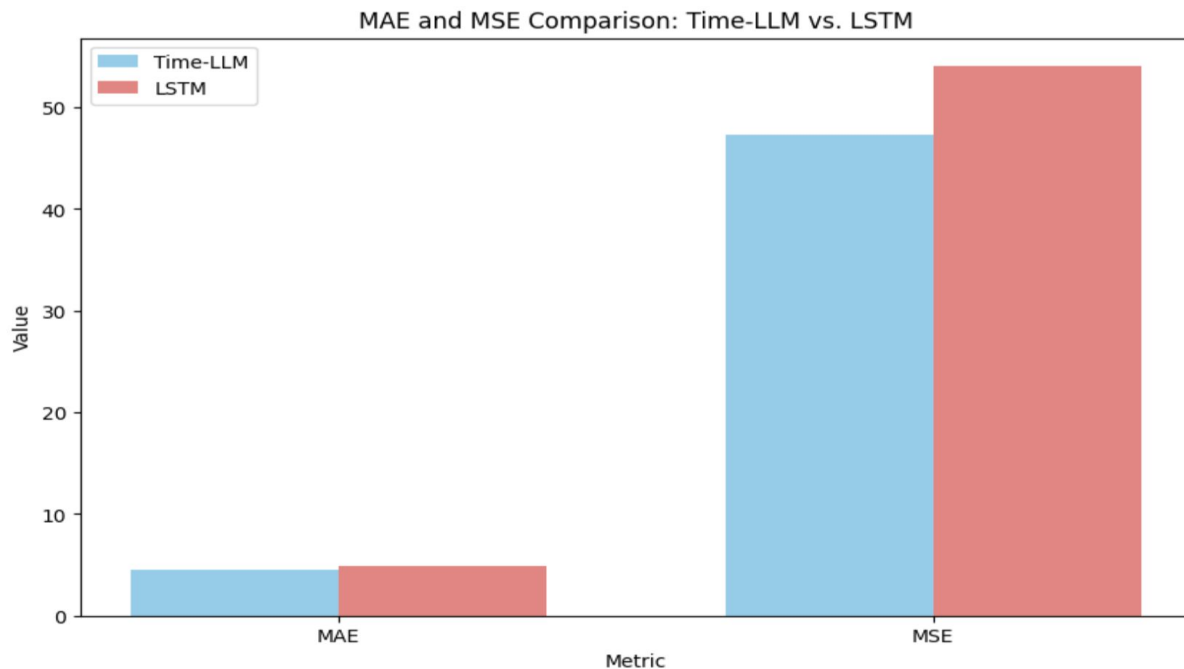
MSE:

- 12.6% reduction in MSE
- 1.14x better than LSTM model

MAE:

- 6.3% reduction in MSE
- 1.07x better than LSTM model

Results & Analysis



Results & Analysis

Table 2: Classification Performance

Model	Class	Precision	Recall	F1-Score	Support
Time-LLM	Outage	1.00	0.58	0.74	24
	Anomaly	0.62	0.93	0.74	14
LSTM	Outage	0.94	0.62	0.75	24
	Anomaly	0.48	0.93	0.63	14

Time-LLM eliminates all false outage alerts and cuts anomaly false positives by roughly 40%, while lifting overall macro-F1 by five points. The only negative is a small 0.04 drop in outage recall, which can likely recover by tuning the outage threshold or adding a learned classifier head.

Results & Analysis

Example LLM prompt and response #1

Example at Test Timestep 0 (True: Normal)

Input (last timestep of sequence):

Bandwidth: 49.23 Mbps

Latency: 14.88 ms

Packet Loss: 0.06%

Predicted Bandwidth: 46.05 Mbps

Actual Bandwidth: 49.11 Mbps

Absolute Error: 3.06 Mbps

Time-LLM Decision: Normal

Example LLM prompt and response #2

Example at Test Timestep 40 (True: Anomaly)

Input (last timestep of sequence):

Bandwidth: 55.00 Mbps

Latency: 18.64 ms

Packet Loss: 0.12%

Predicted Bandwidth: 49.38 Mbps

Actual Bandwidth: 80.10 Mbps

Absolute Error: 30.72 Mbps

Time-LLM Decision: Anomaly detected

Results & Analysis

Example LLM prompt and response #3

Example at Test Timestep 2296 (True: Outage)

Input (last timestep of sequence):

Bandwidth: 82.74 Mbps

Latency: 19.75 ms

Packet Loss: 0.14%

Predicted Bandwidth: 69.23 Mbps

Actual Bandwidth: 0.00 Mbps

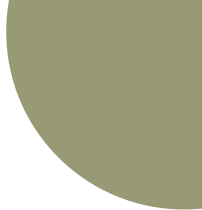
Absolute Error: 69.23 Mbps

Time-LLM Decision: Anomaly detected

Results & Analysis



Overall, Time-LLM cuts regression error by 12.6 % (MSE) and 6.3 % (MAE) over a tuned 2-layer LSTM, eliminates all false outage alerts, and slashes anomaly false positives by roughly 40 %, with only a modest increase in training time.



07

CONCLUSION

Conclusion

Our study has demonstrated that reprogrammed LLMs (DistilGPT2), when applied to time-series forecasting in cybersecurity, can be a robust alternative to traditional LSTM models.

Performance

- 12.6% lower MSE, 6.3% lower MAE and a 0.05 gain in macro-f1 on event detection compared to the LSTM.
- +0.05 improvement in macro-F1 for event classification
- ~40% reduction in false positives and elimination of false outage alerts

Limitations

- Slight drop in outage recall (0.62 → 0.58)

Considerations For Improvement

- Fine-tune detection threshold for better recall
- Testing with smaller and diverse datasets to validate generalisability

Conclusion

Improved Detection Accuracy

- Higher precision and F1-score on anomaly and outage classes
- Better discrimination between normal and abnormal patterns

Contextual Awareness

- Unlike LSTMs, LLMs leverage broader context through prompt-based instruction and prefix engineering, allowing them to detect subtle shifts and previously unseen threats

Generalisation and Efficiency

- The LLM generalized well even with limited training data
- Achieved this with only a modest increase in training time over a 2-layered LSTM

Future Works

- Explore more powerful models such as Mistral and Gemma
- Enhance prompt engineering strategies and tokenization methods
- Extend to real-time, multi-domain anomaly detection settings



THANKS!

Daniel, Tony, Nicholas, Ethan

41079 Computer Science Studio 2

REFERENCES

Tang, H., Zhang, C., Jin, M., Yu, Q., Wang, Z., Jin, X., Zhang, Y., & Du, M. (2024). Time series forecasting with LLMs: Understanding and enhancing model capabilities. arXiv. <https://arxiv.org/pdf/2402.10835>

Jin, M., Wang, S., Ma, L., Chu, Z., Zhang, J. Y., Shi, X., Chen, P. Y., Liang, Y., Li, Y. F., Pan, S., & Wen, Q. (2024). TIME-LLM: Time series forecasting by reprogramming large language models. International Conference on Learning Representations (ICLR). arXiv. <https://arxiv.org/pdf/2310.01728>

Chang, C., Wang, W. Y., Peng, W. C., & Chen, T. F. (2024). LLM4TS: Aligning pre-trained LLMs as data-efficient time-series forecasters. arXiv. <https://arxiv.org/pdf/2308.08469>

Google. (n.d.). Google Colaboratory Sign Up. <https://colab.research.google.com/signup>

Sudhakar, V. M. (2025, March 11). *The LLM advantage: Smarter time series predictions with less effort*. DZone. <https://dzone.com/articles/llm-advantage-smarter-time-series-predictions>

MarcoPeix. (n.d.). *Time Series Analysis – TimeLLM.ipynb* [Jupyter Notebook]. GitHub. <https://github.com/marcopeix/time-series-analysis/blob/master/TimeLLM.ipynb>

REFERENCES

KimMeen. (n.d.). *TimeLLM: Time series forecasting by reprogramming large language models – TimeLLM.py* [Python code]. GitHub.

<https://github.com/KimMeen/Time-LLM/blob/main/models/TimeLLM.py>

Patel, A. A., & Jaiswal, I. (2024, February 29). Comparative Analysis: Gemma 7B vs. Mistral 7B. Ankursnewsletter.com; Ankur's Newsletter. <https://www.ankursnewsletter.com/p/comparative-analysis-gemma-7b-vs>

Talk on MLLM. (2024, April 14). [MLLM Talk] Time-LLM: Time Series Forecasting by Reprogramming Large Language Models [English/英文] [Video]. YouTube. Retrieved May 22, 2025 https://youtu.be/6sFiNExS3nl?si=73Hjfs9rn_zoiUGe

White, J., Fu, Q., Hays, S., Sandborn, M., Olea, C., Gilbert, H., Elnashar, A., Spencer-Smith, J., & Schmidt, D. C. (2023). *A prompt pattern catalog to enhance prompt engineering with ChatGPT*. arXiv. <https://arxiv.org/pdf/2302.11382>

Zhou, W., Liu, X., Zhang, S., Zhang, M., Liu, L., & Ma, Y. (2024). *A survey on prompt engineering for large language models*. In Z. Wang & H. Liu (Eds.), *Artificial Intelligence and Smart Applications* (pp. 327–343). Springer. https://link.springer.com/chapter/10.1007/978-981-99-7962-2_30